“EcoFleet AI: Intelligent Electric Vehicle Fleet Monitoring and Route Optimization Platform”

GIT & DOCKER TECHNICAL ASSIGNMENT

Submitted in the partial fulfilment for the Course of

CSA1012-SOFTWARE ENGINEERING

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE(AI)

Submitted by

P L OAVIKSHA (192572001)

Under the Supervision of

Dr.ANITA DAVAMANI K

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

AUGUST 2026

1. ABSTRACT

EcoFleet AI is a full-stack intelligent electric vehicle fleet management platform designed to improve the monitoring, optimization and operational management of electric vehicle fleets. The system addresses challenges related to battery health, energy consumption, charging management, route planning and predictive maintenance.The proposed platform integrates a web-based monitoring interface with a Flask backend, MySQL database, AI/ML prediction modules, GPS-based route visualization and Docker-based deployment. Vehicle and sensor information is collected and processed to provide fleet managers with real-time operational information.Machine learning is incorporated to predict battery degradation and maintenance requirements, while optimization techniques are used to recommend suitable charging schedules and energy-efficient routes. Git is used for version control, branching and collaborative development, while Docker provides a consistent and portable deployment environment.The architecture is designed to provide modularity, maintainability, scalability, reliability and portability, making EcoFleet AI suitable as a foundation for intelligent and sustainable EV fleet operations.

2. INTRODUCTION

The increasing adoption of electric vehicles has created new challenges for fleet operators. Unlike conventional vehicles, EV fleets require continuous monitoring of battery state, charging requirements, energy consumption and vehicle availability.Conventional fleet systems generally concentrate on vehicle location and basic tracking. However, modern EV fleet management requires intelligent decision-making capabilities that can predict battery degradation, identify maintenance requirements, optimize charging and recommend efficient routes.EcoFleet AI addresses these challenges through an integrated software platform combining IoT data, GPS information, artificial intelligence, machine learning, database management and web technologies.The system provides fleet managers with a centralized dashboard to monitor vehicle conditions, analyze fleet performance and make data-driven operational decisions.

3. PROBLEM ANALYSIS

3.1 Existing Problem

EV fleet operators must simultaneously manage:

- Vehicle availability
- Battery health
- Charging schedules
- Energy consumption
- Maintenance
- Vehicle locations
- Route planning

- Charging-station availability

Managing these activities manually or using conventional tracking systems can result in:

- Poor battery utilization
- Unplanned maintenance
- Excessive charging costs
- Inefficient routes
- Vehicle downtime
- Delivery delays
- Increased operational complexity

3.2 Proposed Solution

EcoFleet AI provides a centralized intelligent platform that:

Monitor → Analyze → Predict → Optimize → Alert → Report

The system continuously processes fleet information and converts it into useful operational recommendations.

4. SYSTEM REQUIREMENT ANALYSIS

4.1 Functional Requirements

FR1 – Authentication

The system shall authenticate users and provide role-based access.

FR2 – Vehicle Management

The system shall allow fleet managers to add, update, remove and monitor EVs.

FR3 – Battery Monitoring

The system shall display battery percentage, battery health, temperature, voltage and charging cycles.

FR4 – GPS Tracking

The system shall display vehicle locations using an interactive map.

FR5 – Charging Management

The system shall monitor charging stations and charging sessions.

FR6 – Battery Prediction

The system shall predict battery degradation using historical vehicle data.

FR7 – Predictive Maintenance

The system shall identify vehicles requiring maintenance.

FR8 – Route Optimization

The system shall recommend energy-efficient routes.

FR9 – Analytics

The system shall generate fleet-performance analytics.

FR10 – Alerts

The system shall generate alerts for abnormal vehicle conditions.

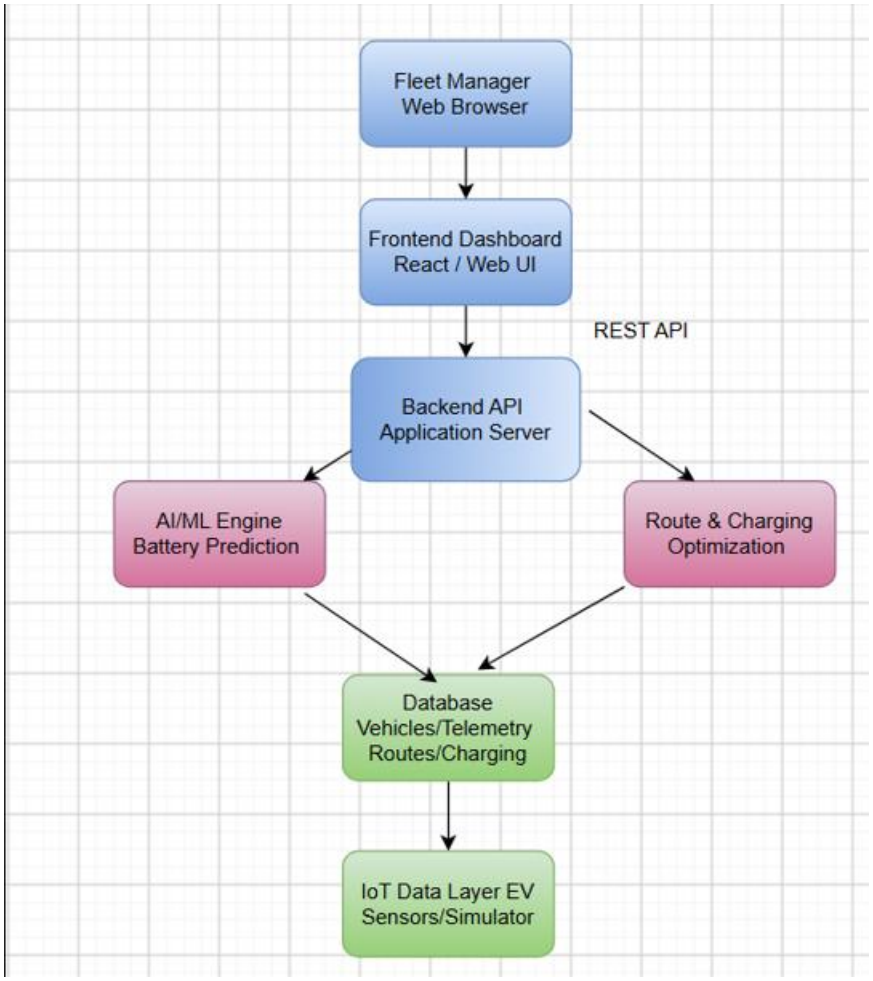
FR11 – Reports

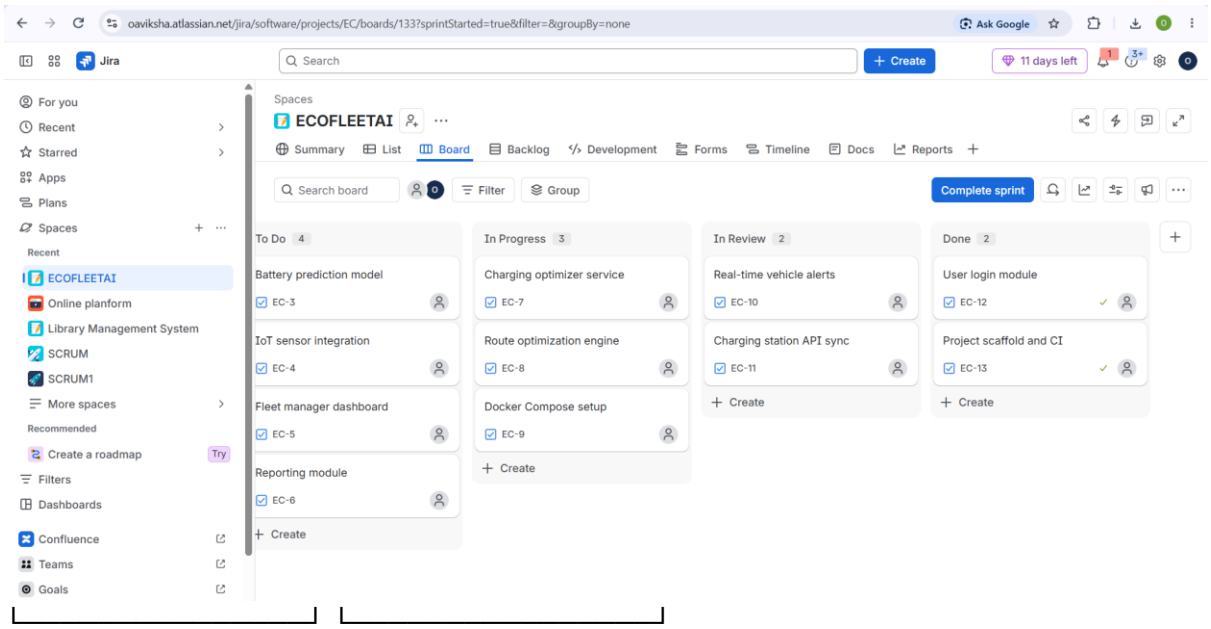
The system shall generate operational reports.

5. NON-FUNCTIONAL REQUIREMENTS

Attribute	Requirement
Performance	Dashboard should respond quickly
Security	User and database access must be protected
Scalability	Additional vehicles should be supported
Reliability	Data should be stored consistently
Maintainability	Modules should be independently maintainable
Portability	Application should run consistently using Docker
Usability	Dashboard should be simple and responsive
Availability	System should remain accessible during operation

6. SYSTEM ARCHITECTURE





7. WHY THIS ARCHITECTURE?

This is where you can gain marks for **architectural justification**.

Separation of concerns

Each layer performs a specific responsibility.

Maintainability

Changes in the UI do not directly affect database implementation.

Scalability

Additional AI modules and APIs can be introduced later.

Security

Users communicate with the database through the backend rather than directly.

Reusability

The Flask REST API can later support a mobile application.

Portability

Docker can package the application and database environment.

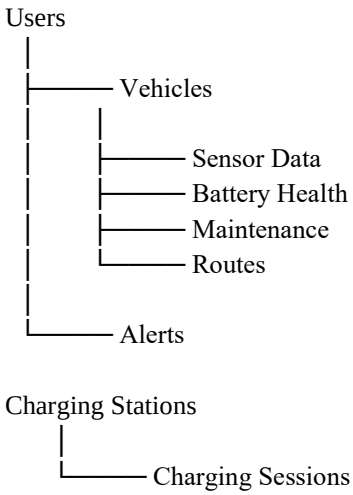
8. COMPONENT DESIGN

Component	Responsibility	Technology
User Interface	Fleet dashboard	HTML/CSS/JS
Authentication	Login and access control	Flask
Fleet Manager	Vehicle CRUD	Flask + MySQL
Sensor Processor	Process EV data	Python
Battery AI	Battery prediction	Scikit-learn
Maintenance AI	Failure-risk prediction	Python ML
Charging Optimizer	Charging recommendations	Python
Route Optimizer	Efficient routes	Python + Leaflet
Analytics Engine	Fleet statistics	Flask + Chart.js
Notification Engine	Alerts	Flask
Database	Persistent storage	MySQL

Component	Responsibility	Technology
Containerization	Deployment	Docker

9. DATABASE COMPONENT

(Main database entities:)



10. AI/ML COMPONENT

This is what makes your project different from a normal fleet CRUD application.

Battery Prediction

Historical Data
↓
Data Cleaning
↓
Feature Selection
↓
ML Model
↓
Battery Health Prediction
↓
Risk Classification
↓
Maintenance Recommendation

Input Features

- Battery age
- Charging cycles
- Temperature
- Energy consumption
- Distance travelled
- Average charging level
- Current battery health

Output

Predicted Battery Health
↓
Degradation Level
↓
Maintenance Risk

11. ROUTE OPTIMIZATION

The route module considers:

- Distance
- Travel time
- Battery level
- Energy consumption
- Charging stations

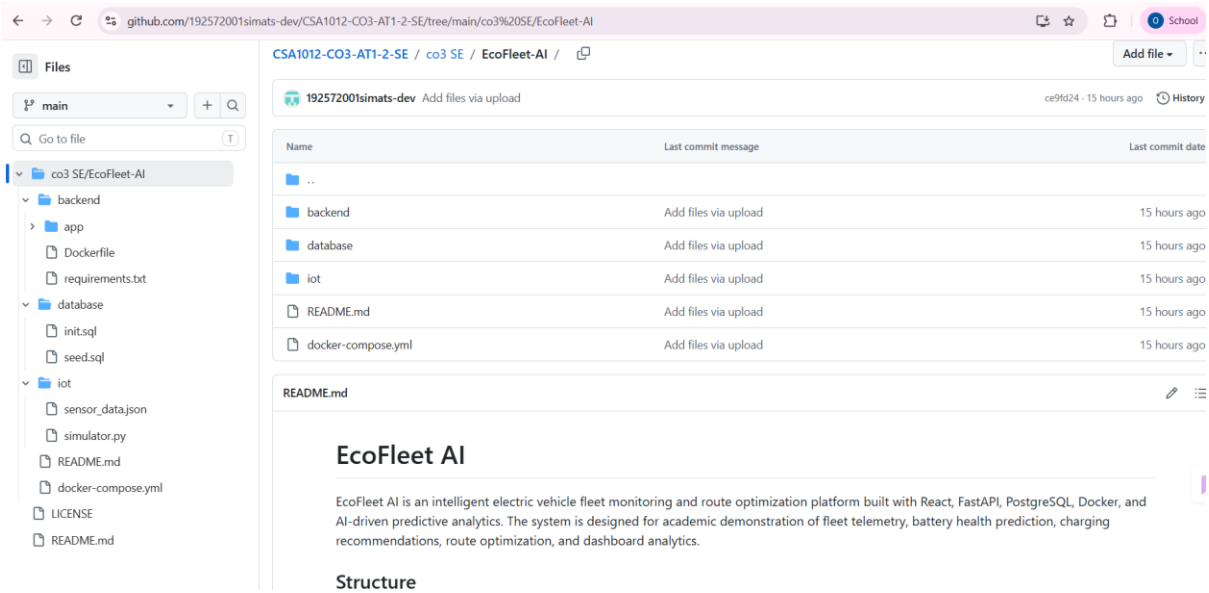
The system compares possible routes and recommends the most suitable route.

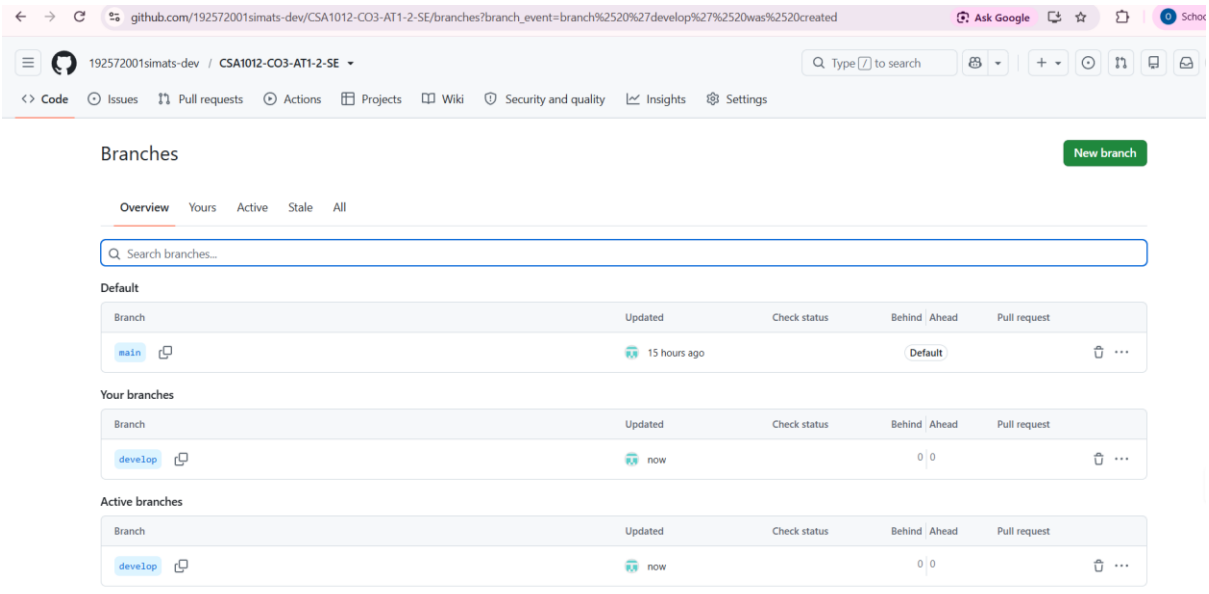
Start
↓
Retrieve Vehicle Data
↓
Check Battery
↓
Generate Candidate Routes
↓
Calculate Energy Requirement
↓
Check Charging Stations
↓
Calculate Route Score
↓
Recommend Best Route

12. GIT ARCHITECTURE

The assignment specifically requires repository initialization, branching strategy, workflow and commit history.

main
└── development
    ├── feature/dashboard
    ├── feature/battery-ai
    ├── feature/charging
    ├── feature/route
    └── feature/maintenance





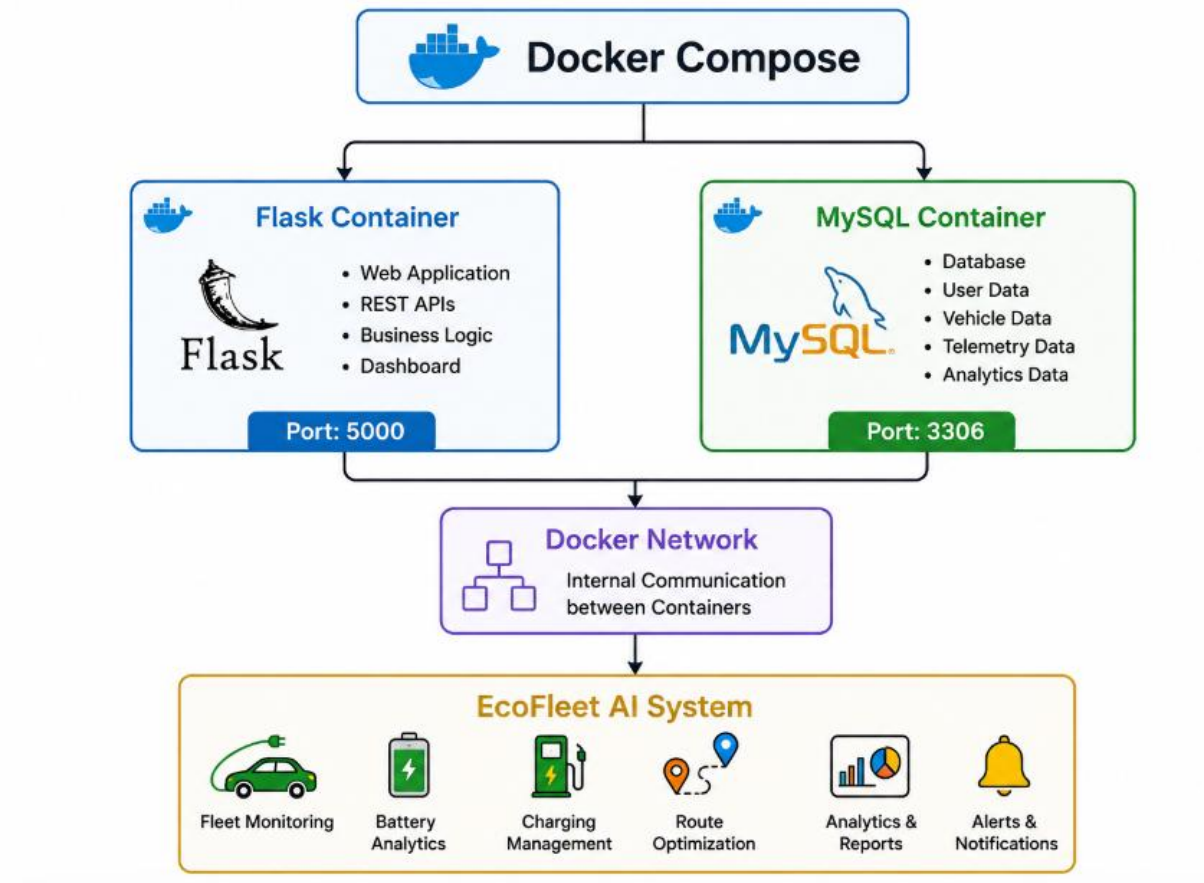
Example Git Workflow

```
git init
git add .
git commit -m "Initial project setup"

git checkout -b development
git checkout -b feature/battery-ai

git add .
git commit -m "Implement battery prediction"

git push origin feature/battery-ai
```

After testing:

```
git checkout development
git merge feature/battery-ai
git push origin development
```

13. DOCKER ARCHITECTURE



The official assessment expects a Dockerfile and Docker Compose configuration with services, networking, volumes, environment variables and dependencies.

14. DOCKERFILE

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

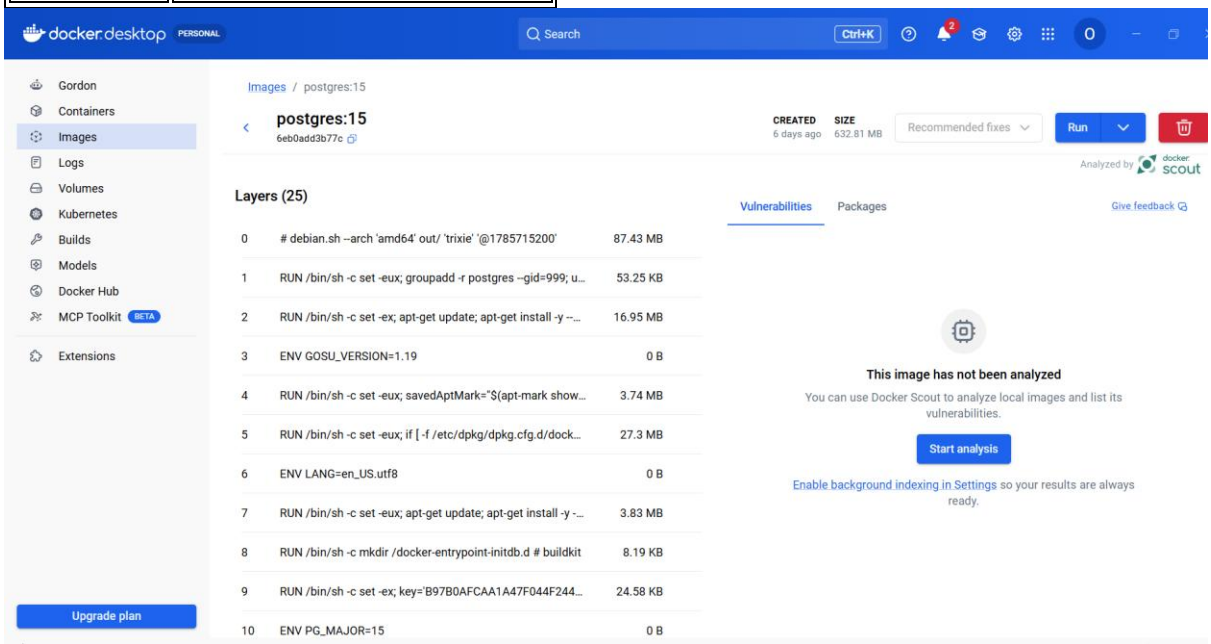
COPY . .

EXPOSE 5000

CMD ["python", "app.py"]

Technical Explanation

Instruction	Purpose
FROM	Provides Python runtime
WORKDIR	Defines application directory
COPY	Copies required files
RUN	Installs dependencies
EXPOSE	Defines application port
CMD	Starts Flask application



15. DOCKER COMPOSE

```
services:

  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      DB_HOST: db
      DB_USER: root
      DB_PASSWORD: root
      DB_NAME: ecofleet
    depends_on:
      - db

  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: ecofleet
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  mysql_data:
```

16. DEPLOYMENT AND TESTING

The application is deployed using:

```
docker compose build
docker compose up -d
docker ps
```

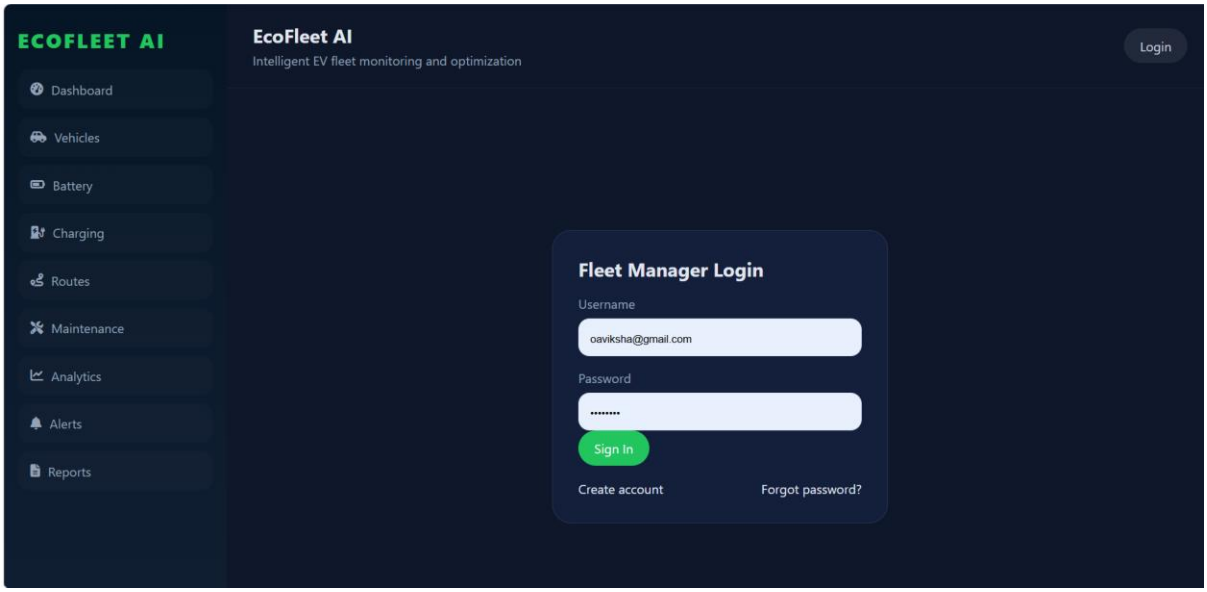
Application:

```
http://localhost:5000
```

Testing should cover:

- Login
- Dashboard
- Vehicle management
- Battery monitoring
- AI prediction
- Charging
- Route optimization
- Maintenance

- Analytics
- Database connectivity
- Docker containers



.

17. RESULTS

After implementation, EcoFleet AI is expected to provide:

Operational Results

- Centralized fleet monitoring
- Improved vehicle visibility
- Faster fleet-manager decisions

AI Results

- Battery degradation prediction
- Maintenance-risk identification
- Charging recommendations
- Route recommendations

Technical Results

- Version-controlled source code
- Reproducible Docker environment
- Modular architecture
- Portable deployment

18. CHALLENGES AND SOLUTIONS

Challenge	Solution
Limited real EV sensor data	Simulated IoT dataset
Limited battery dataset	Sample historical dataset
Real-time GPS dependency	Leaflet + simulated coordinates
Docker database connection	Docker service networking
Dependency conflicts	Docker environment
Collaborative development	Git branching
AI prediction accuracy	Future real-world dataset training

19. CONCLUSION

EcoFleet AI provides a complete intelligent solution for electric vehicle fleet management by integrating web technologies, AI/ML, GPS, IoT data, database management, Git and Docker.The proposed three-tier architecture separates presentation, application and data responsibilities while the AI layer provides intelligent prediction and optimization capabilities.The use of Git provides structured version control and enables feature-based development, while Docker provides a consistent and portable execution environment.The system therefore addresses the major operational challenges of EV fleet management and provides a scalable foundation for future smart transportation applications